CSCI160 Computer Science I
Lab Exercise: Loops, Conditional Logic, and Sequence Generation

---

Objectives:

- Follow instructions carefully, sticking with the current module's topics
- Practice writing loops (while and for loops)
- Implement conditional logic and input validation
- Work with mathematical sequences and iterative problem-solving
- Write clean, readable, and well-documented Python code
- Trace code execution in the debugger
- Cite code accessed through AI or Internet resource

NOTE: If you use any code or techniques that are not included in our current lesson or suggested by AI, tutor or any Internet source, you ***must*** cite and ***thoroughly explain in comments*** even if you did write the code straight off the top of your head.

Failure to do so will result in an automatic 20% point deduction. No exceptions. If you are unsure, reach out for clarification.

RESTRICTIONS: No functions (def statements) are allowed

Lab Problems:

Problem 1: Collatz Conjecture (Hailstone Sequence)

$$f(n) = \begin{cases} \frac{n}{2} & if\ n\ is\ even \\ 3n+1 & if\ n\ is\ odd \end{cases}$$

- Create Python source file called ***hailstone.py***. Include your authorship header and brief description of the problem.
- Prompt the user for a positive integer
- Using a ***while loop and compound boolean expression***, validate that the input is numeric and greater than 0 (invalid entries print `X IS INVALID` and re-prompt). Allow the user infinite attempts to enter correct input.
- Calculate and print the hailstone sequence (Collatz sequence) for the number
- The Collatz conjecture (also known as the ***3n + 1 problem***, or the ***3n + 1 conjecture***) is a conjecture in mathematics that concerns a numeric sequence algorithm defined as follows:
    - Start with any positive integer ***n***.
    - Then ***each following term*** is obtained from the previous term as follows:
        - If the current term is even, ***the next term is one half of the term.***
        - If the current term is odd, ***the next term is 3 times the term plus 1***.
- The conjecture is that no matter what initial value of n, the sequence will always reach 1. So n becoming 1 will be the termination point of your while loop

- The sequence of numbers involved is sometimes referred to as the **hailstone sequence** or hailstone numbers (because the values are usually subject to multiple descents and ascents like hailstones in a cloud)
- Print the total number of elements in the sequence on the same line, separated by a space.

Sample Run:

```
Enter a number: seventeen
seventeen is invalid
Enter a number: -17
-17 is invalid
Enter a number: 17

Hailstone sequence for: 17
17 52 26 13 40 20 10 5 16 8 4 2 1
Hailstone sequence for 17 has 13 elements
```

Problem 2: Factor Finder, prime numbers, perfect numbers and the aliquot sum

- Factors of ***n*** are the set of integers ***{1, 2 . . ., n}*** that evenly divide ***n***
- Create Python source file called ***factors.py***. Include your authorship header and brief description of the problem.
- Using a ***while loop and compound boolean expression***, validate that input is numeric and greater than 0. Allow the user 3 attempts to enter correct input. If they fail to do so in 3 attempts, print an error message and shut the program down.
- Using a ***for loop with a range of n + 1***, find all factors of the given number and add them to a list. Include both ***1 and n*** as factors
- Using a separate ***for each loop list traversal,*** print the factors from the list all on the same line without the brackets or commas normally seen when printing lists
- Print the total count of factors. You can use ***len()*** for this
- Print a message that states if the number is prime or composite.

    - A number is prime if the only factors are 1 and itself (total count of 2 factors)
    - Otherwise it is composite

- Using a separate ***for loop with a range of n – 1***, calculate the ***aliquot sum of n***. The aliquot sum is the sum of the proper divisors of ***n***, that is, all divisors excluding the number itself.

    - The aliquot sum for 10 is ***1 + 2 + 5 = 8***
    - Do not use Python's sum function for this. Write the logic yourself.

- Print a message stating whether ***n*** is a perfect number. In number theory a ***perfect number*** is a positive integer that is equal to it's ***aliquot sum***

Sample Runs:

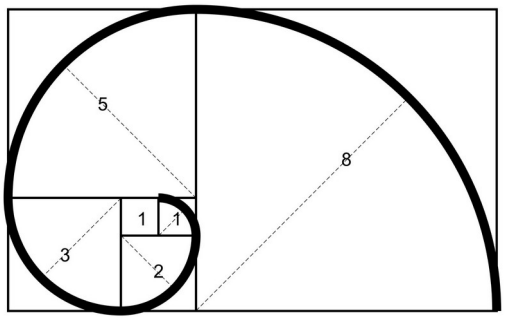
```
Enter a positive integer: 28

Factors of 28: 1 2 4 7 14 28
Number of factors: 6
28 is a composite number
Aliquot sum for 28 is 28
28 is a perfect number

Enter a positive integer: -11
Enter a positive integer: eleventy
Enter a positive integer: 0
Enter a positive integer: 11

Factors of 11: 1 11
Number of factors: 2
11 is a prime number
Aliquot sum for 11 is 1
11 is an imperfect number
```

Problem 3: Fibonacci Sequence Generator



- Create Python source file called ***fibonacci.py***. Include your authorship header and brief description of the problem.
- Prompt the user for a positive integer
- Using a ***while loop and compound boolean expression***, validate that the input is numeric and greater than 0 (invalid entries print `X IS INVALID` and re-prompt). Allow the user infinite attempts to enter correct input.
- Starting at 0 and 1, print the ***Fibonacci sequence*** up to that number of terms
- In mathematics, the ***Fibonacci sequence*** is a sequence in which each element is the sum of the two elements that precede it. Numbers that are part of the Fibonacci sequence are known as ***Fibonacci numbers***, commonly denoted F_n
- Many writers begin the sequence with 0 and 1, although some start it from 1 and 1 and some (as did Fibonacci) from 1 and 2. Starting from 0 and 1, the sequence begins
    - 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144

Sample Run:

```
Enter number of terms: 10
Fibonacci sequence: 0 1 1 2 3 5 8 13 21 34

Enter number of terms: 2
Fibonacci sequence: 0 1
```

Problem 4: Integer digit counting, reversal and palindromes

- **Restrictions:** The input for this problem ***must be treated as an integer***. You are not allowed to use Python's built in ***len()*** or String reversal functions. This is an iterative mathematical problem for you to practice looping logic. Please treat it as such.
- Create Python source file called ***digits.py***. Include your authorship header and brief description of the problem.
- Prompt the user for an integer ***n***
- Using a ***while loop and compound boolean expression***, validate that input is numeric and greater than 0. Allow the user 4 attempts to enter correct input. If they fail to do so in 3 attempts, print an error message and shut the program down.
- Using a while loop and a counter variable, count the number of digits in ***n.*** The loop should continue as long as ***n > 0***
- Inside the loop you can use ***integer division*** to ***remove the last digit.***

    - If `num = 12345`, then `12345 // 10 = 1234` (removes the last digit `5`).
    - If `num = 1234`, then `1234 // 10 = 123` (removes the last digit `4`).

- The loop repeats the above steps until `num` becomes 0. Here's how the variables change in each iteration:

Iteration	`num` (before division)	`num // 10` (after division)	`count`
1	12345	1234	1
2	1234	123	2
3	123	12	3
4	12	1	4
5	1	0	5

- Once you have that complete modify the loop body to also reverse the number. You can use modulus division to ***access the last digit*** prior to removing it
- Create a new variable called ***reversed_num*** to hold your new value.
- With each iteration use modulus division to access the last digit.
- Use addition to build ***reversed_num***. Remember to multiply by 10 to move numbers over to hold each new number. This must be done mathematically, not using string behaviors.

Iteration	`num`	`num % 10`	`reversed_num`
1	12345	5	5
2	1234	4	54
3	123	3	543
4	12	2	5432
5	1	1	54321

- Once the number has been reversed, print a message stating whether or not the number is a palindrome. (A palindrome is a sequence that is identical forwards and backwards)

Sample Run:

```
Enter a number: 12345
Number of digits: 5
Reversed: 54321

Enter a number: 1234321
Number of digits: 7
Reversed: 1234321
1234321 is a palindrome.

Enter a number: 19283738291
Number of digits: 11
Reversed: 19283738291
19283738291 is a palindrome.
```

Deliverables:

- Submit four Python files: `collatz.py`, `factors.py`, `fibonacci.py`, and `digits.py`
- Each file should contain:
    - A proper header comment with your name, date, and a brief problem description
    - Clear and consistent white space and commenting
    - Citations and detailed explanations of any code not written by the author (AI, tutor, Internet . . . etc).

30-Point Rubric:

Category	Points
Input validation works correctly for all problems	4
Correct and efficient loop usage, loop structures match the instructions	10
Output matches requested format and examples	6
Proper variable naming and descriptive comments	5
Code runs without errors	5
Total	**30**
